

Research Note: Low-Level Design (LLD) Practice & Evaluation

Project: LLD Practice Platform
Author: Engineering Team
Date: September 2026
Assignment Context: 2-Day Engineering Challenge

1. Executive Summary & Problem Space

Low-Level Design (LLD) — also known as Object-Oriented Design (OOD) or Machine Coding — sits at the critical intersection of software architecture and daily programming. Unlike High-Level Design (HLD), which focuses on distributed systems, data pipelines, caching, and network topologies, LLD focuses on **classes, interfaces, object responsibilities, state transitions, design patterns, maintainability, and clean code principles (SOLID)**.

While candidates and practicing engineers have an abundance of resources to learn LLD concepts, **evaluating and practicing LLD remains deeply broken**.

The Core Learner Dilemma

When an engineer practices Data Structures and Algorithms (DSA), the feedback loop is immediate and deterministic:

- *Input* $\rightarrow$ *Output* $\rightarrow$ *Pass/Fail (Time & Space Complexity)*.

In contrast, when an engineer designs a Parking Lot, Elevator System, or Vending Machine:

- 1. No Single "Correct" Answer:** Multiple valid abstractions exist. Should pricing be calculated using the Strategy pattern, Decorator pattern, or a simple rule table? Is spot allocation handled by the Lot, a Level, or an independent Dispatcher?
- 2. Ambiguity Between Working Code vs. Good Design:** A learner can write 200 lines of functional code that passes basic scenarios, yet completely violates the Single Responsibility Principle (SRP) and Open/Closed Principle (OCP). Conversely, a learner may over-engineer with 15 interfaces for a trivial problem.
- 3. Lack of Actionable, Explainable Critique:** Without a senior staff engineer looking over their shoulder, learners are left asking:
 - *"Did I pick the right abstractions?"*
 - *"Is my design extensible if new requirements appear?"*
 - *"Are my classes tightly coupled?"*
 - *"How does my solution compare to an industry-standard implementation?"*

2. Research of Existing Approaches & Market Tools

To understand where the current learning ecosystem fails, we analyzed four primary approaches available to learners today:

Approach / Tool	How It Works	Key Strengths	Critical Gaps in LLD Practice
LeetCode / HackerRank	Solves algorithmic puzzles with automated test cases.	Instant pass/fail execution, automated test benches, global leaderboards.	Zero architectural feedback. Encourages anti-patterns (e.g., packing all logic into a single God method <code>solve()</code> with global static variables) just to pass tests.
Generic LLM Chat (ChatGPT / Claude)	Learner pastes code or ideas and asks: <i>"Is this good LLD?"</i>	Highly conversational, flexible, accessible.	Inconsistent rubrics and hallucinations. Praise bias (tends to say "Looks great!") without rigorous critique), lack of deterministic contract verification, and no structured attempt tracking across iterations.

Books & Static Courses (e.g., Alex Xu, Head First Design Patterns, Grokking OOD)	Case-study walkthroughs of classic problems (Parking Lot, Movie Ticket Booking).	Canonical solutions, clean diagrams, clear pattern explanations.	Passive reading with zero interactive practice. Learners suffer from the "illusion of competence" — understanding a solution in print is vastly different from designing one from a blank canvas.
Peer / Senior Code Reviews	Manual review on GitHub PRs or mock interview platforms (Pramp, Interviewing.io).	High contextual fidelity, deep human nuance, discussable trade-offs.	Expensive, unscalable, and slow. Turnaround time is hours or days; review quality varies wildly depending on the reviewer's personal stylistic biases.

3. Key Gaps Identified

From our user research and competitive review, four structural gaps emerged:

Gap 1: The Multi-Modal Submission Void

In actual engineering and interviews, LLD is not just raw source code. It consists of:

- **Domain Entities & Class Contracts** (Attributes, methods, visibility, inheritance, interfaces).
- **Design Rationale & Assumptions** (e.g., "Why did I choose Strategy over State? What are my concurrency assumptions?").
- **Behavioral Walkthrough** (How entities collaborate during a primary flow).

Existing coding platforms only accept a single code file, discarding the architectural rationale. Diagram tools (Miro, Lucidchart) capture diagrams without verifiable contracts.

Gap 2: Binary Grading vs. Multi-Dimensional Rubrics

Treating an LLD solution as binary (Pass/Fail) is fundamentally flawed. A meaningful assessment must break design into orthogonal dimensions:

1. **Requirements Completeness:** Did the solution address all core business rules?
2. **SOLID & Design Patterns:** Are responsibilities segregated? Can new behaviors be extended without modifying existing classes?
3. **Coupling & Extensibility:** Are dependencies inverted through abstractions, or are concrete classes hard-wired?
4. **Resilience & Edge Cases:** What happens on state overflows (lot full, payment failure, invalid state transitions)?

Gap 3: Absence of the "What-If" Stress Test

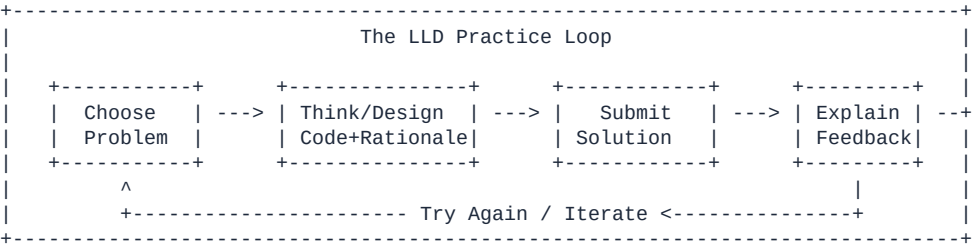
Great system design interviews test flexibility: "What if we need to add electric charging spots with dynamic hourly billing?" Existing automated tools never challenge the learner with downstream requirement changes to test if their abstractions actually hold up.

Gap 4: Broken Iteration History

Learners rarely design a perfect system on attempt 1. Improvement requires an explicit **practice loop**: seeing how Attempt 2 refactored the God Class from Attempt 1, and tracking rubric score progression over time.

4. Product Direction & Core Principles

To address these gaps within a focused 2-day assignment scope, we designed the **LLD Practice Platform** around five guiding tenets:



1. Dual-Track Evaluation: Deterministic AST Rules + Semantic LLM Reasoning

- **Deterministic Engine:** Fast, objective structural checks using AST analysis. Checks whether required domain entities exist, detects God Classes (methods > threshold), measures interface usage, and flags direct concrete instantiations violating Dependency Inversion (DIP).
- **Semantic Engine (LLM / Heuristic Expert):** Evaluates design intent, trade-off rationale, elegance of pattern selection, and synthesizes personalized constructive critique.
- **Fail-Safe Fallback:** If LLM evaluation is unavailable, slow, or times out, the deterministic engine automatically synthesizes an explainable report. The learner is never left hanging.

2. Multi-Modal Artifact Submission

A learner provides:

- **Code Workspace:** Core classes, interfaces, and methods in clear typed code.
- **Design Rationale & Assumptions:** Explicit text explaining pattern choices and trade-offs.
- **Live Visual Representation:** Interactive class diagram auto-rendered in real-time (via Mermaid.js) so the learner visually inspects class relationships while coding.

3. Rubric-Grounded, Explainable Feedback

Feedback is structured into four transparent categories (0–100 score + severity badges: Critical, Warning, Suggestion, Praise). Every criticism comes with:

- **Why it matters** (e.g., SRP violation causes high test fragility).
- **Concrete Refactoring Nudge** (e.g., "Extract `FeeCalculator` into a `PricingStrategy` interface").
- **"What-If" Architectural Challenge** (e.g., "How would your design accommodate valet parking?").

4. Progression & Attempt Diffing

Each problem attempt is versioned. The learner can review their attempt history, observe score trajectory across rubrics, and inspect exact code diffs between iterations to build tangible confidence.

5. Conclusion

By shifting from binary algorithmic execution to multi-dimensional, rubric-driven architectural evaluation, the LLD Practice Platform solves the core dilemma of LLD learners. It transforms ambiguous self-doubt into a structured, explainable, and repeatable path toward engineering mastery.